COS120 - CẤU TRÚC DỮ LIỆU VÀ GIẢI THUẬT

BÀI 7: CÂY NHỊ PHÂN TÌM KIẾM CÂN BẰNG

MỤC TIÊU

- Hiểu rõ cây nhị phân tìm kiếm cân bằng (gọi tắt là cây AVL)
- Hiện thực được các thao tác cơ bản của cây AVL.
- Hiểu và thực hiện được các tác vụ quay cây để có thể giúp cho cây cân bằng trở lại.
- Hiểu được ưu điểm và hạn chế của cây AVL so với cây BST.

NỘI DUNG

- 7.1 - CÂY NHỊ PHÂN TÌM KIẾM CÂN BẰNG - AVL
- 7.2 - CÁC TÁC VỤ XOAY CÂY
- 7.3 - HIỆN THỰC CÂY AVL
- 7.4 - BÀI TẬP CÂN BẰNG CÂY AVL

CÂY NHỊ PHÂN TÌM KIẾM CÂN BẰNG

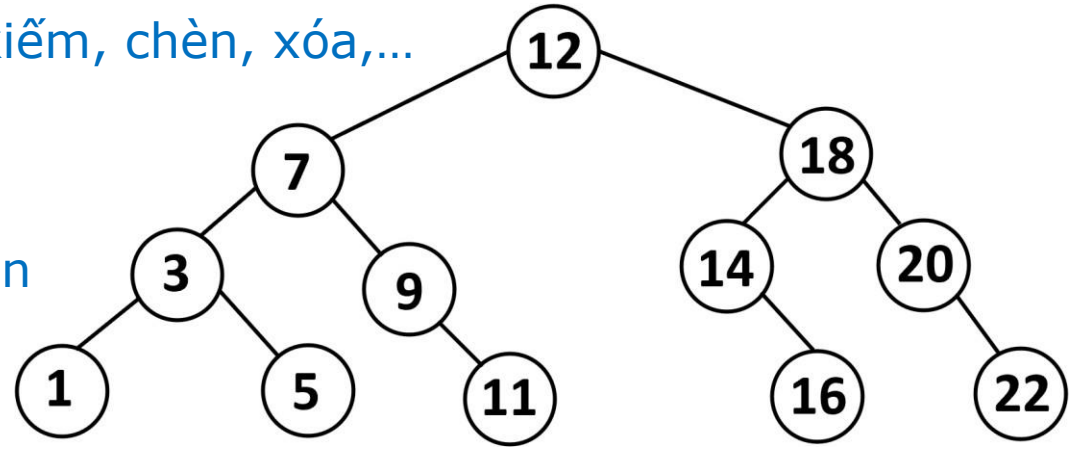
7.1 - CÂY NHỊ PHÂN TÌM KIẾM CÂN BẰNG - AVL

CÂY NHỊ PHÂN CÂN BẰNG HOÀN TOÀN

KHÁI NIỆM VÀ HIỆU QUẢ CỦA CÂY CÂN BẰNG HOÀN TOÀN

Cây nhị phân cân bằng hoàn toàn là cây BST trong đó:

- Tại mỗi nút, số nút của cây con trái và cây con phải chênh lệch không quá một.
- Giúp tối ưu hóa về hiệu suất cho các thao tác tìm kiếm, chèn, xóa,…



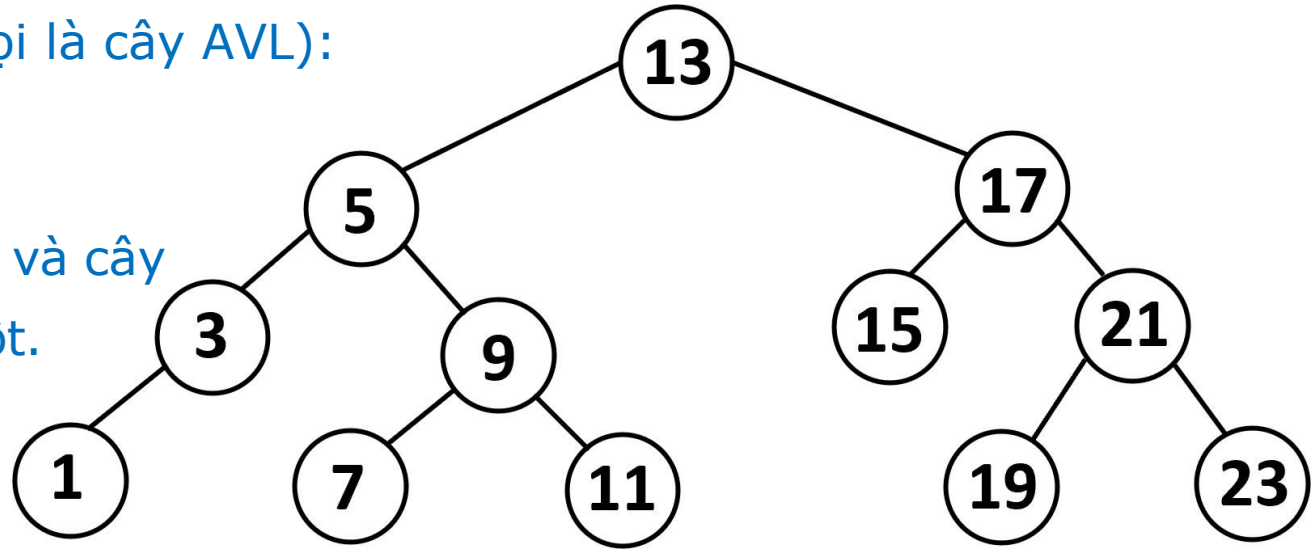
HẠN CHẾ CỦA CÂY CÂN BẰNG HOÀN TOÀN

- Một cây khó đạt được trạng thái cân bằng hoàn toàn
- Rất dễ mất cân bằng khi thêm hoặc hủy các nút.
- Việc cân bằng lại cây tốn kém vì phải thao tác trên toàn bộ cây.

→ Cần tổ chức một cây nhị phân cân bằng yếu hơn → **đó là sự ra đời của cây AVL**

KHÁI NIỆM CÂY AVL

- Cây nhị phân tìm kiếm cân bằng (hay còn gọi là cây AVL):
  - Là một loại cây nhị phân tìm kiếm.
  - Tại mỗi nút, chiều cao của cây con trái và cây con phải chỉ chênh lệch **không** quá một.
- Giúp đảm bảo rằng cây luôn cân bằng.
- Tối ưu hóa thời gian tìm kiếm, chèn, và xóa các phần tử.
- AVL là viết tắt của "Adelson-Velsky và Landis", tên của ba nhà khoa học Nga phát minh ra cây BST cân bằng.



MỘT SỐ ƯU ĐIỂM CỦA CÂY AVL

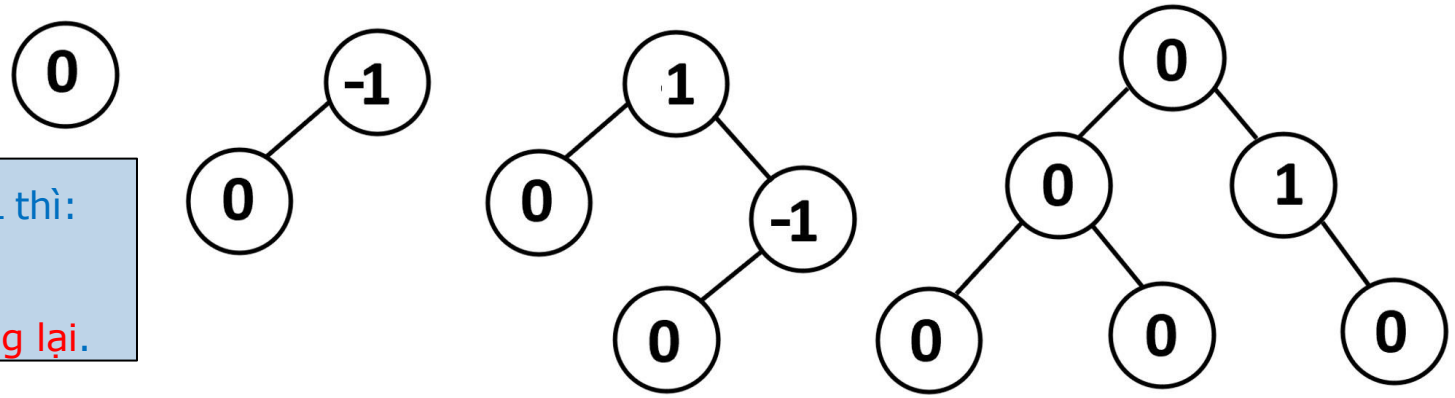
- **Tìm kiếm cân bằng**
  - cây luôn duy trì trạng thái cân bằng, chiều cao tối đa là O(**$\log_2 n$**).
- **Chèn và xóa hiệu quả**
  - chi phí cân bằng lại sau mỗi thao tác chèn/xóa giảm đáng kể.
- **Độ phức tạp đồng nhất**
  - độ phức tạp của chèn, xóa và tìm kiếm **ổn định** ở mức O($\log_2 n$).
- **Dễ dàng triển khai**
  - đơn giản và hiệu quả, được sử dụng rộng rãi.

CHỈ SỐ CÂN BẰNG CỦA MỘT NÚT

- Khi một nút ở trạng thái cân bằng, nó thuộc ba trường hợp:
  - Trường hợp **0**: ta ký hiệu **EH** (cây con trái và phải bằng nhau)
  - Trường hợp **1**: ta ký hiệu **RH** (cây con phải cao hơn)
  - Trường hợp **-1**: ta lý hiệu **LH** (cây con trái cao hơn)

Các ký hiệu này được gọi là **chỉ số cân bằng** của một nút.

Hình vẽ sau đây minh họa **giá trị balFactor** (chỉ số cân bằng) của mỗi nút trong cây AVL.



CHÚ Ý: Sau khi chèn, xóa một nút trong cây AVL thì:

- Kiểm tra nếu cây mất cân bằng.
- Thực hiện các thao tác quay cây để cân bằng lại.

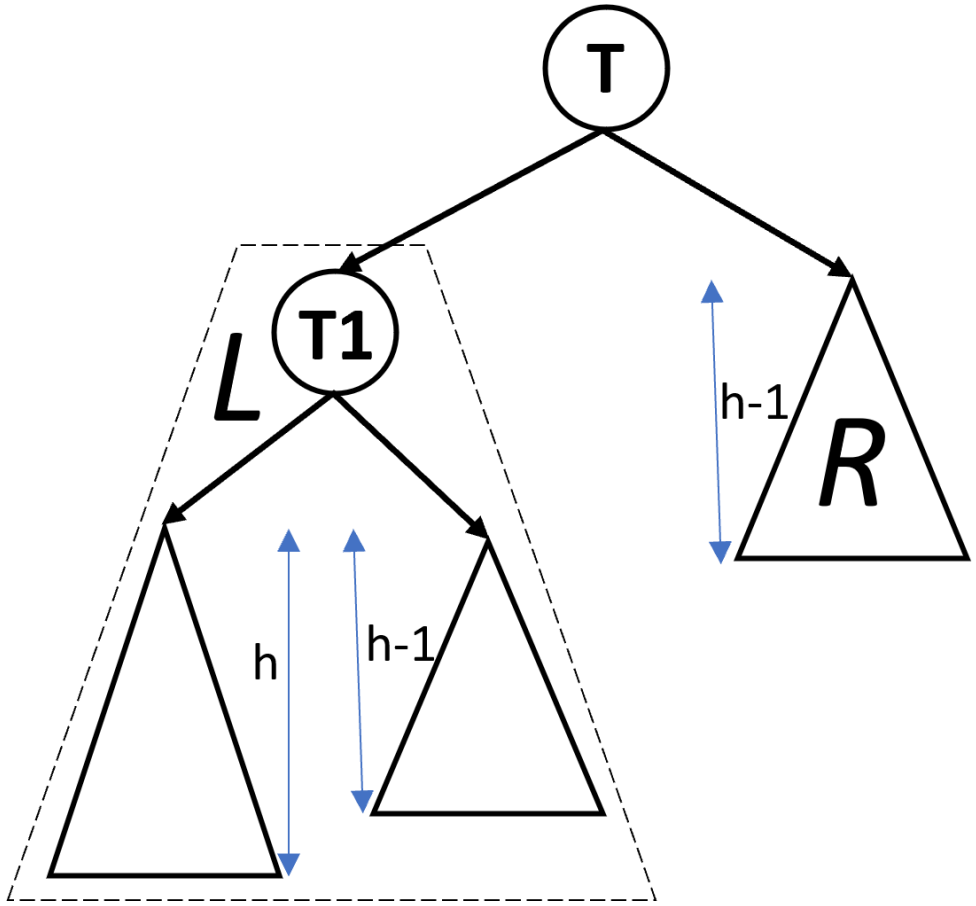
CÂY NHỊ PHÂN TÌM KIẾM CÂN BẰNG

7.2 - CÁC TÁC VỤ XOAY CÂY

TRƯỜNG HỢP 1 – CÂY LỆCH VỀ BÊN TRÁI

TRƯỜNG HỢP 1-1

- Là trường hợp cây mất cân bằng tại T và bị lệch về bên nhánh trái L.
- Nút T1 có chỉ số cân bằng là LH (tức là -1).

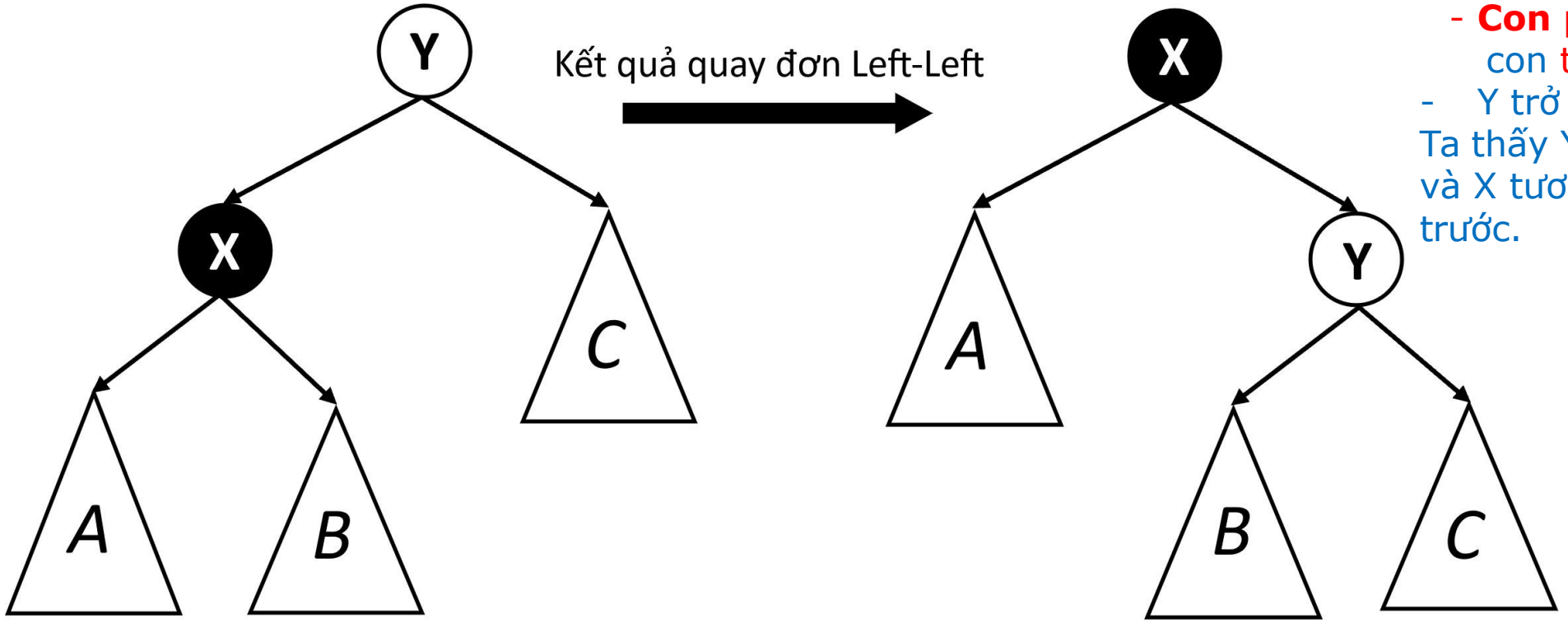


NHẬN XÉT:

- Nhánh L có chiều cao là h+1, nhánh R có chiều cao là h-1. Lệch nhau 2 đơn vị.
- Nhánh L cân bằng, có nút gốc là T1. Tại T1, nhánh trái cao hơn nhánh phải một đơn vị.

TRƯỜNG HỢP 1 – CÂY LỆCH VỀ BÊN TRÁI

QUY TRÌNH QUAY ĐƠN LEFT-LEFT



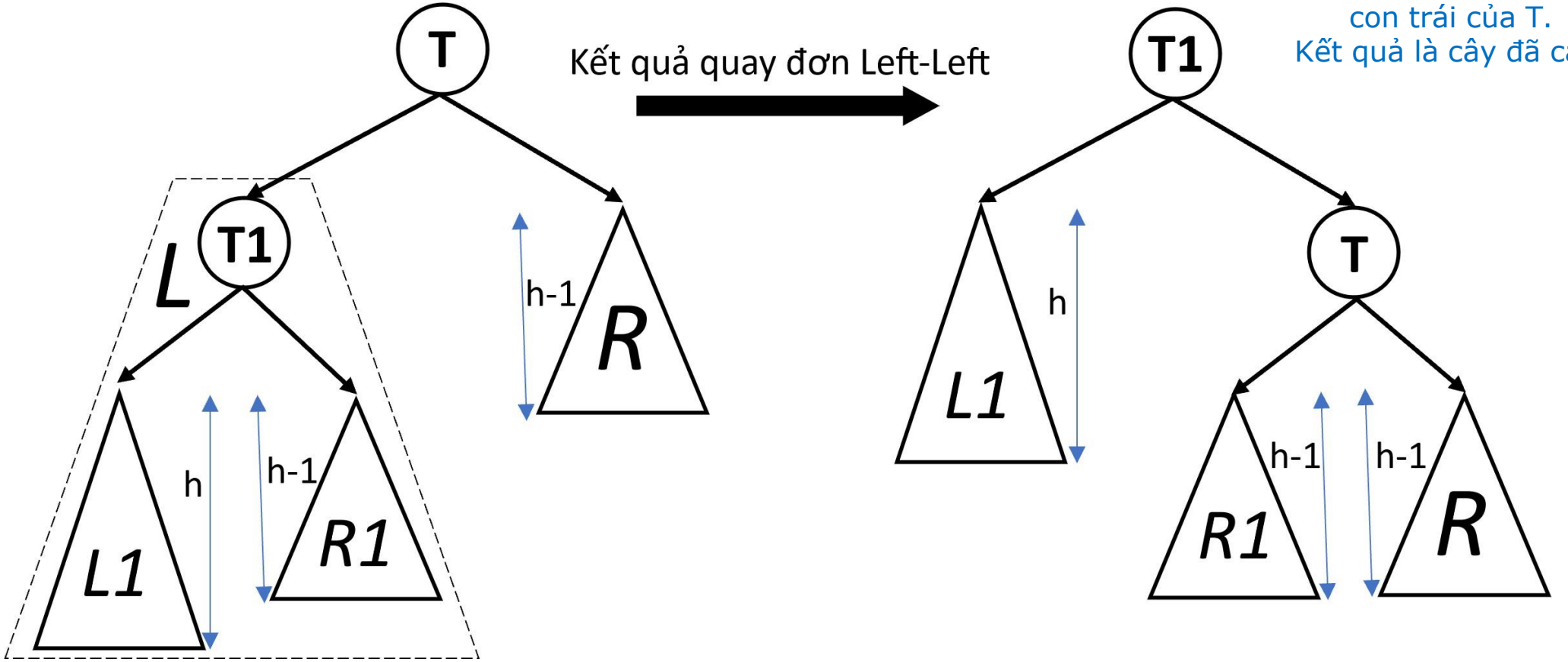
Giả sử mất cân bằng tại ***Y***.
Nút X là con trái của Y.
Thực hiện quay như sau:
- Đặt X làm nút **gốc mới**.
- **Con phải của X** trở thành con trái của Y.
- Y trở thành con phải của X.

Ta thấy Y tương đương T, và X tương đương T1 của slide trước.

TRƯỜNG HỢP 1 – CÂY LỆCH VỀ BÊN TRÁI

CÁCH XỬ LÝ TRƯỜNG HỢP 1-1

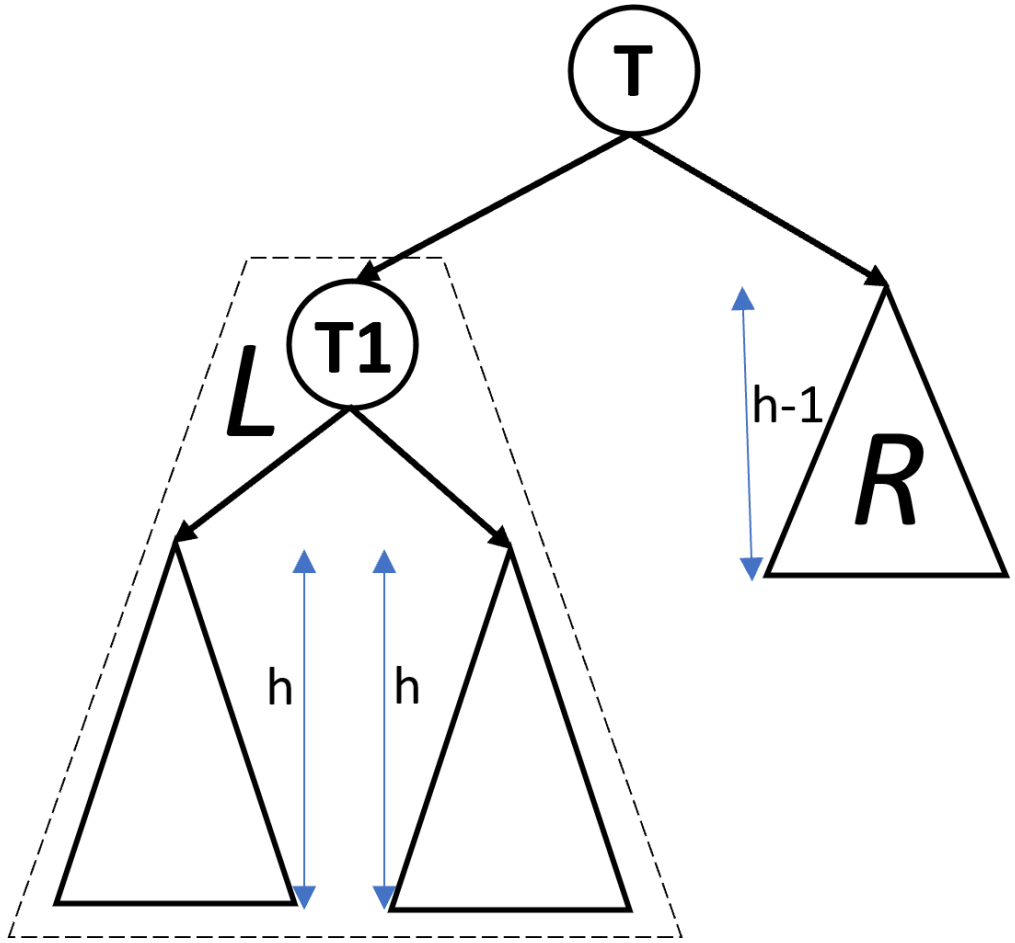
Ta thực hiện phép quay đơn Left-Left.
-Ta đặt nút T1 là nút gốc mới.
-T trở thành con phải của T1.
-Nhánh R1 của T1 trở thành con trái của T.
Kết quả là cây đã cân bằng trở lại.



TRƯỜNG HỢP 1 – CÂY LỆCH VỀ BÊN TRÁI

TRƯỜNG HỢP 1-2

- Là trường hợp cây mất cân bằng tại T và bị lệch về bên nhánh trái L.
- Nút T1 có chỉ số cân bằng là EH (tức là 0).



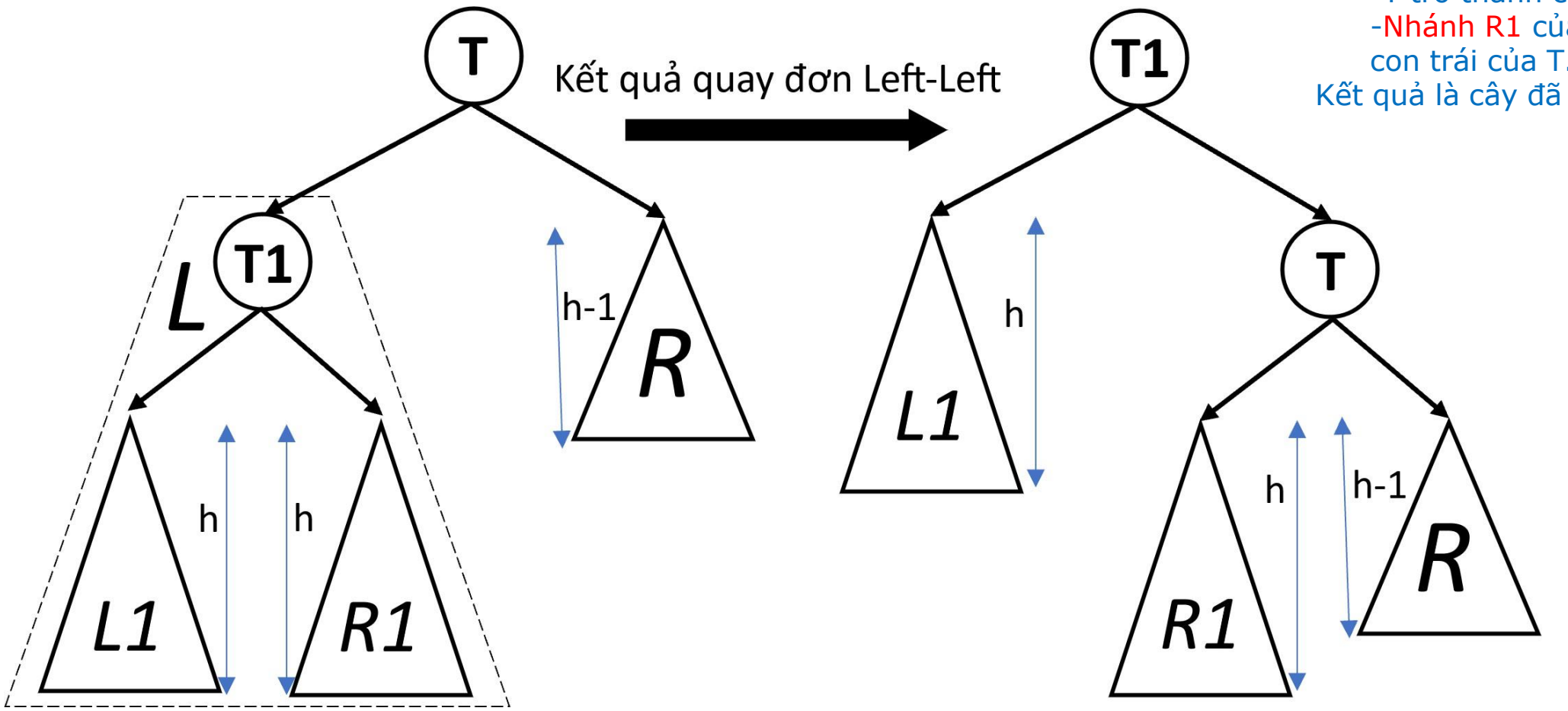
NHẬN XÉT:

- Nhánh L có chiều cao là h+1, nhánh R có chiều cao là h-1. Lệch nhau 2 đơn vị.
- Nhánh L cân bằng, có nút gốc là T1. Tại T1, nhánh trái và nhánh phải bằng nhau.

TRƯỜNG HỢP 1 – CÂY LỆCH VỀ BÊN TRÁI

CÁCH XỬ LÝ TRƯỜNG HỢP 1-2

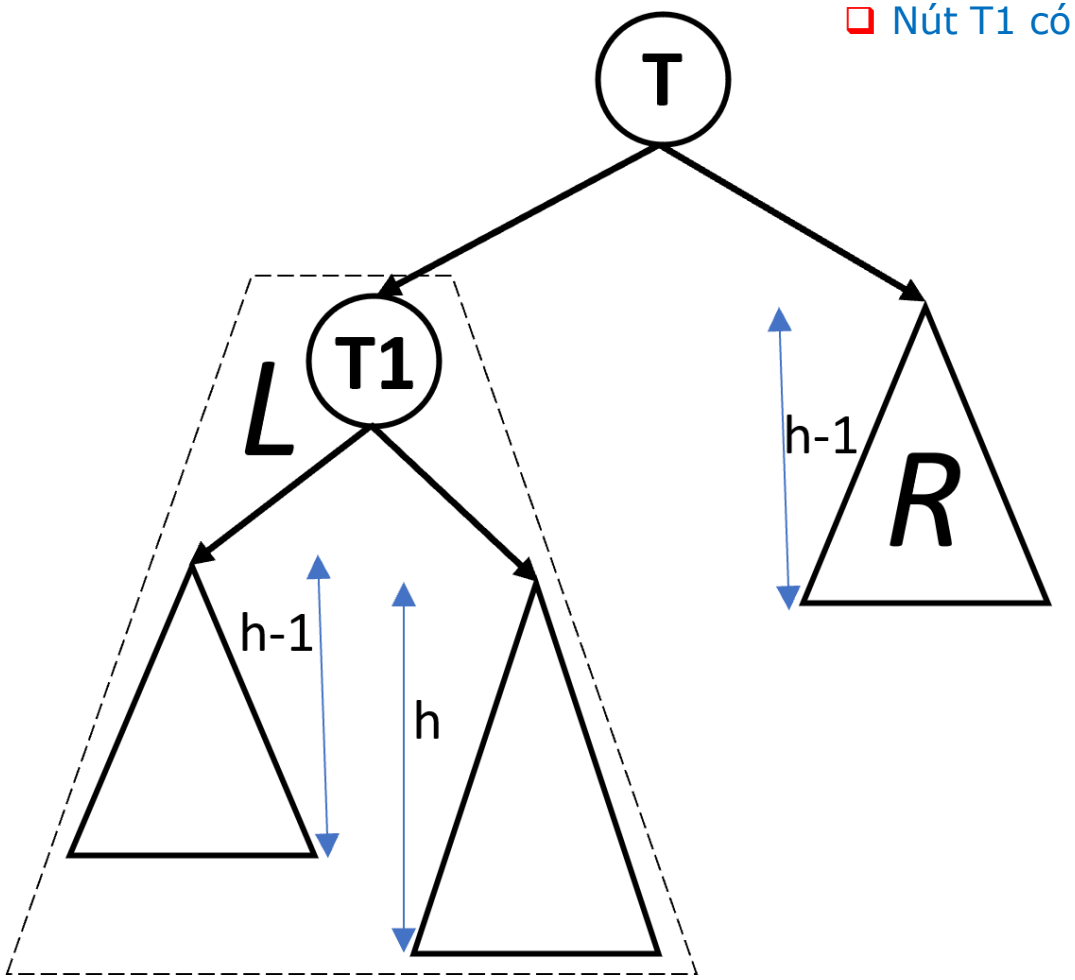
Ta thực hiện phép quay đơn Left-Left.
-Ta đặt nút T1 là nút gốc mới.
-T trở thành con phải của T1.
-Nhánh R1 của T1 trở thành con trái của T.
Kết quả là cây đã cân bằng trở lại.



TRƯỜNG HỢP 1 – CÂY LỆCH VỀ BÊN TRÁI

TRƯỜNG HỢP 1-3

- Là trường hợp cây mất cân bằng tại T và bị lệch về bên nhánh trái L.
- Nút T1 có chỉ số cân bằng là RH (tức là 1).



NHẬN XÉT:

- Nhánh L có chiều cao là h+1, nhánh R có chiều cao là h-1. Lệch nhau 2 đơn vị.
- Nhánh L cân bằng, có nút gốc là T1. Tại T1, nhánh phải cao hơn nhánh trái một đơn vị.

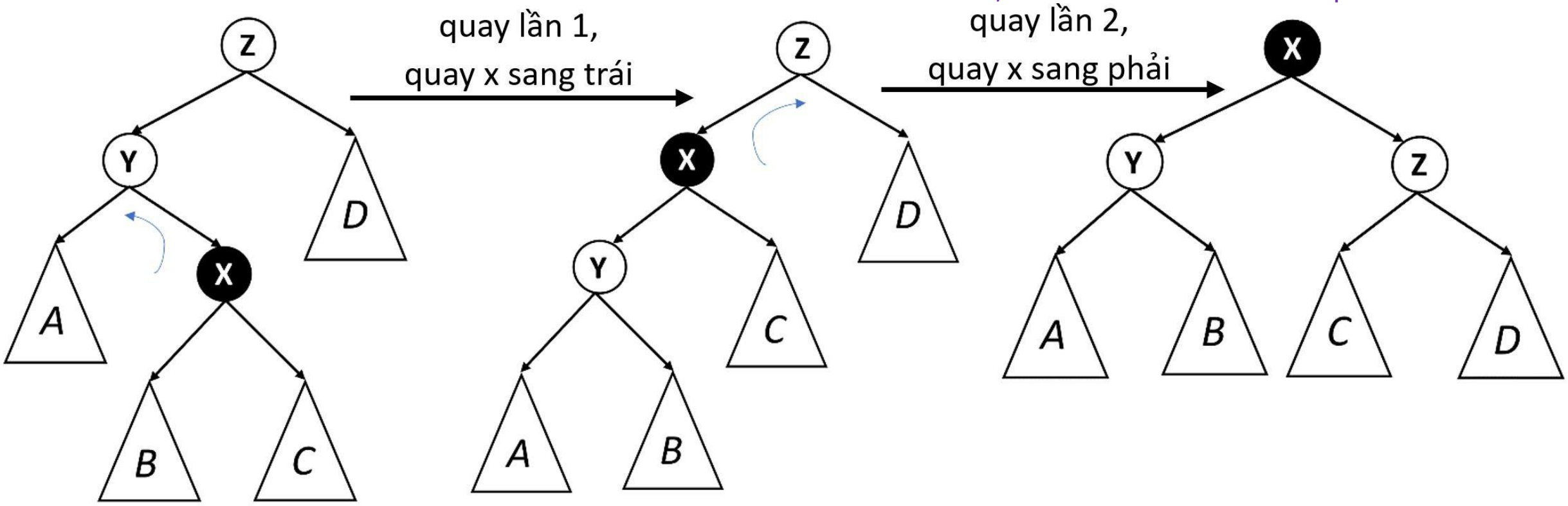
TRƯỜNG HỢP 1 – CÂY LỆCH VỀ BÊN TRÁI

QUY TRÌNH QUAY KÉP LEFT-RIGHT

-Lần quay thứ nhất, quay X từ phải sang trái tại cây Y .

-Lần quay thứ hai, quay X từ trái sang phải tại cây Z.

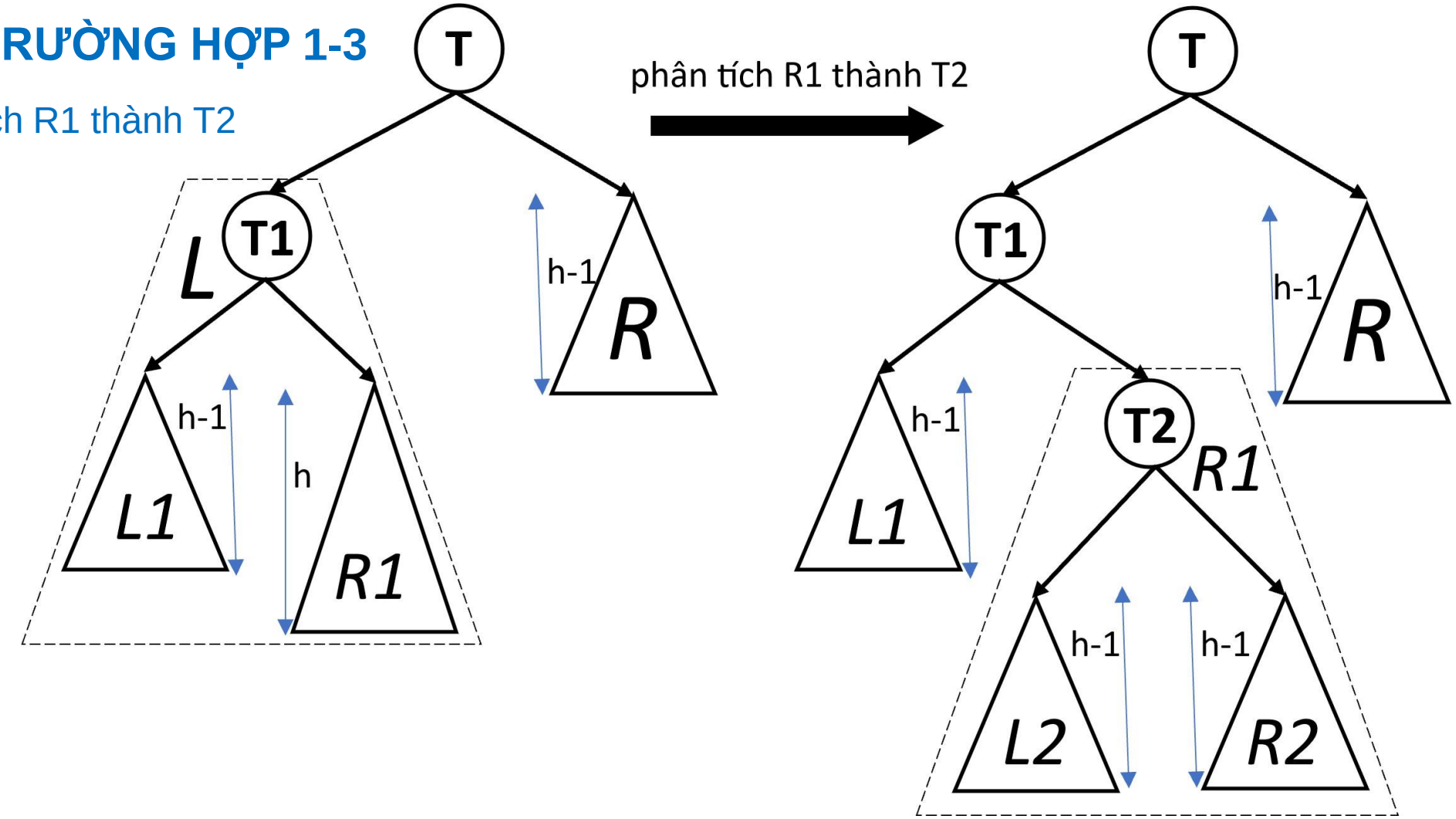
-Kết quả, nút X trở thành nút gốc mới, Y trở thành nhánh trái của X, còn Z trở thành nhánh phải của X.



TRƯỜNG HỢP 1 – CÂY LỆCH VỀ BÊN TRÁI

CÁCH XỬ LÝ TRƯỜNG HỢP 1-3

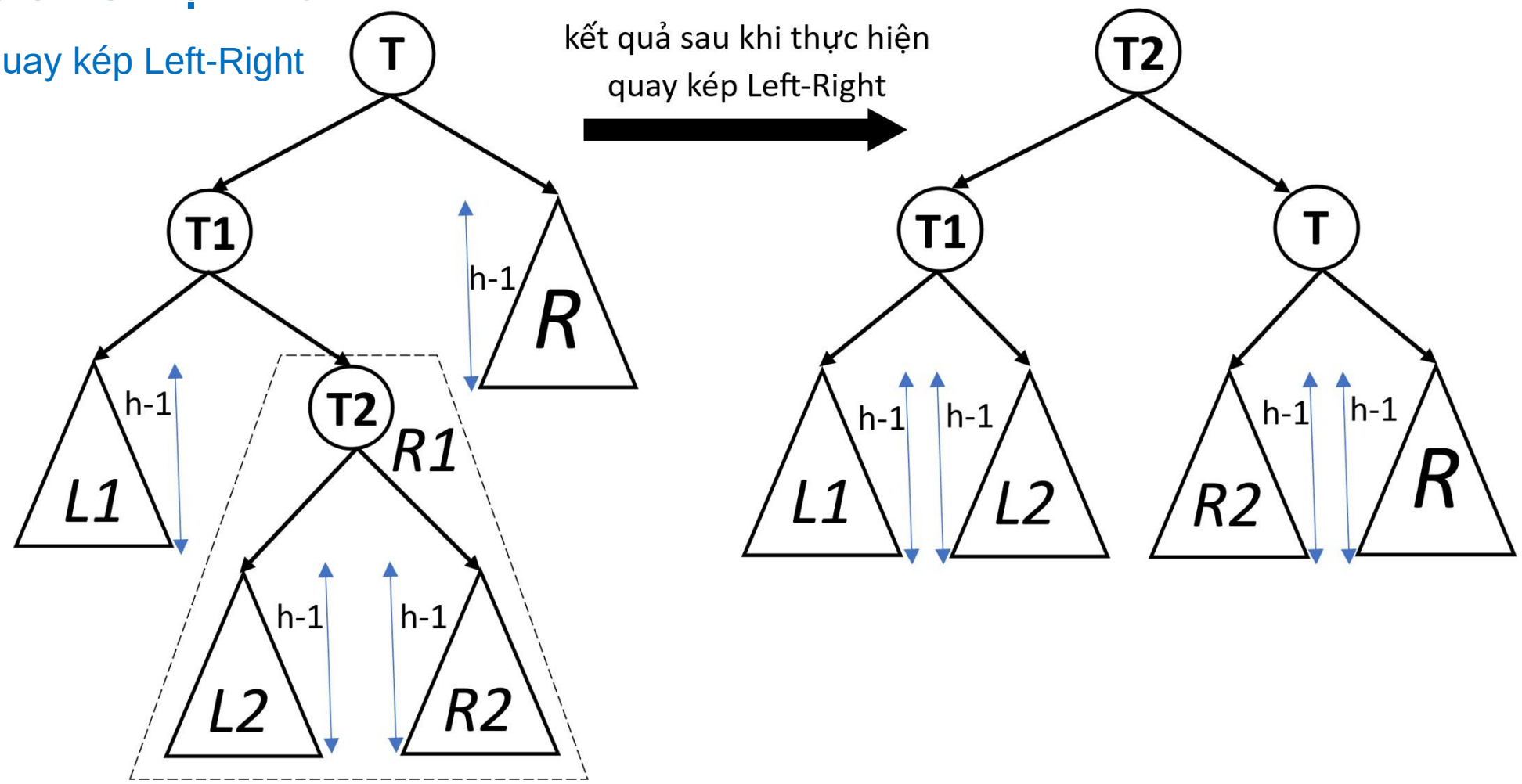
- **Bước 1:** phân tích R1 thành T2



TRƯỜNG HỢP 1 – CÂY LỆCH VỀ BÊN TRÁI

CÁCH XỬ LÝ TRƯỜNG HỢP 1-3

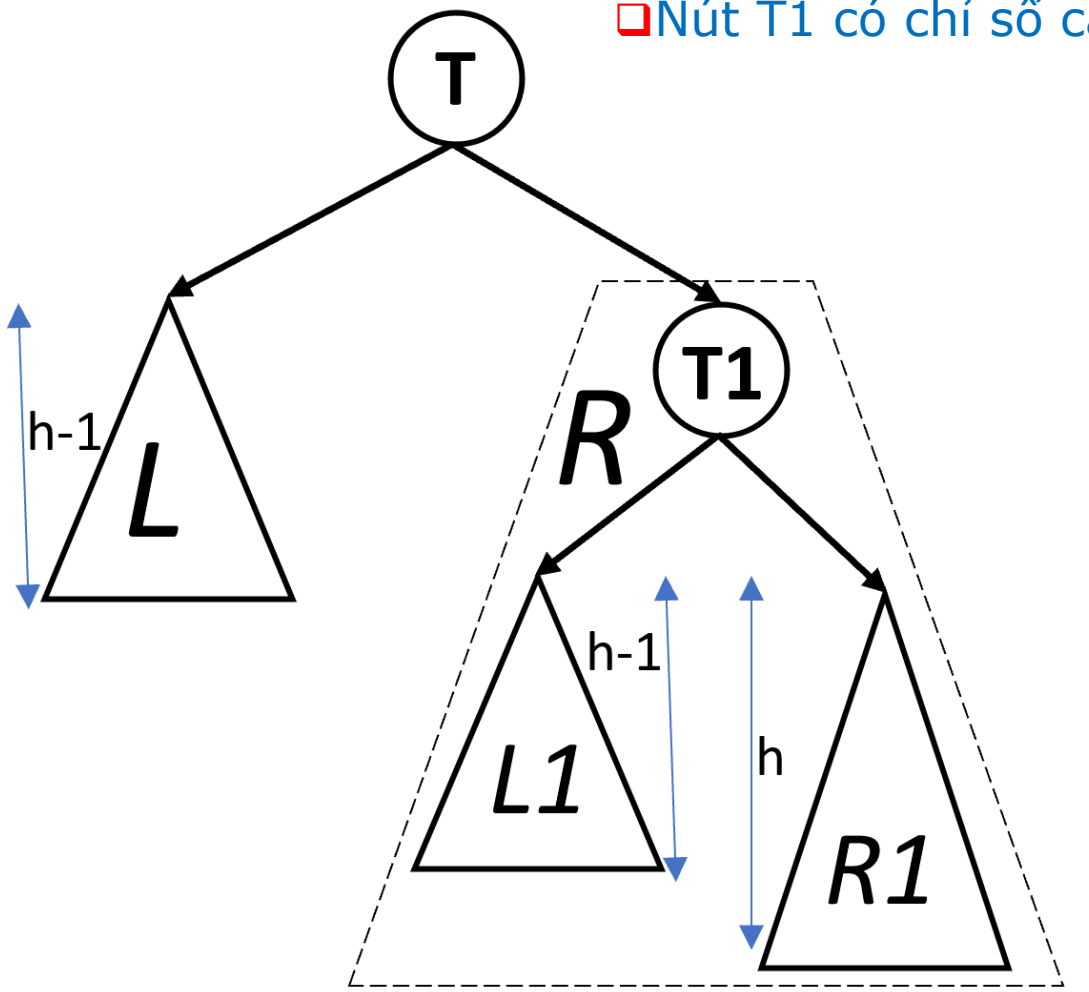
- **Bước 2:** thực hiện quay kép Left-Right



TRƯỜNG HỢP 2 - CÂY LỆCH VỀ BÊN PHẢI

TRƯỜNG HỢP 2-1

- Là trường hợp cây mất cân bằng tại T và bị lệch về bên nhánh phải R.
- Nút T1 có chỉ số cân bằng là RH (tức là 1).



NHẬN XÉT:

- Nhánh R có chiều cao là h+1, nhánh L có chiều cao là h-1. Lệch nhau 2 đơn vị.
- Nhánh R cân bằng, có nút gốc là T1. Tại T1, nhánh phải cao hơn nhánh trái một đơn vị.

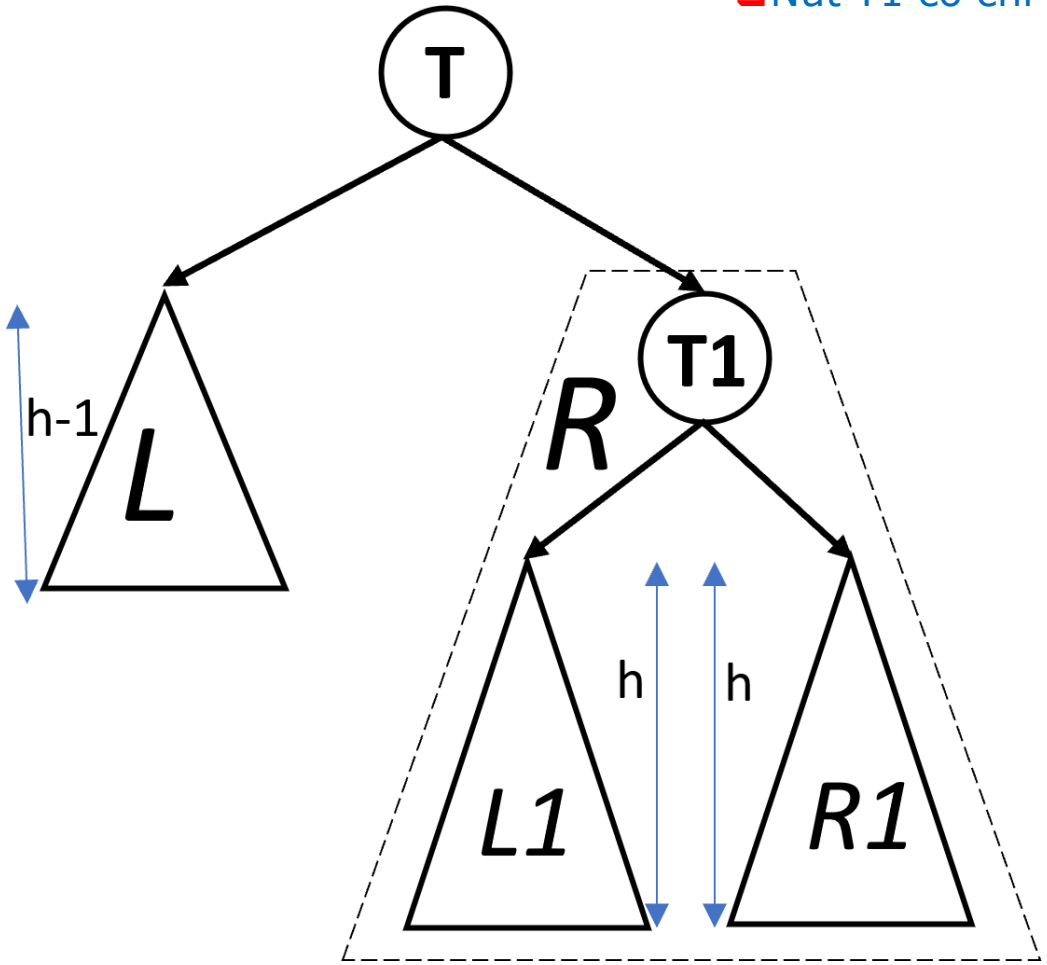
CÁCH XỬ LÝ:

- Trường hợp này là đối xứng của trường hợp 1-1 nói trên.
- Để cân bằng lại cây này, chỉ cần làm các thao tác tương tự trường hợp 1-1 nhưng ngược hướng.
- Cụ thể là thực hiện phép quay đơn Right-Right.

TRƯỜNG HỢP 2 - CÂY LỆCH VỀ BÊN PHẢI

TRƯỜNG HỢP 2-2

- Là trường hợp cây mất cân bằng tại T và bị lệch về bên nhánh phải R.
- Nút T1 có chỉ số cân bằng là EH (tức là 0).



NHẬN XÉT:

- Nhánh R có chiều cao là h+1, nhánh L có chiều cao là h-1. Lệch nhau 2 đơn vị.
- Nhánh R cân bằng, có nút gốc là T1. Tại T1, nhánh phải và nhánh trái bằng nhau.

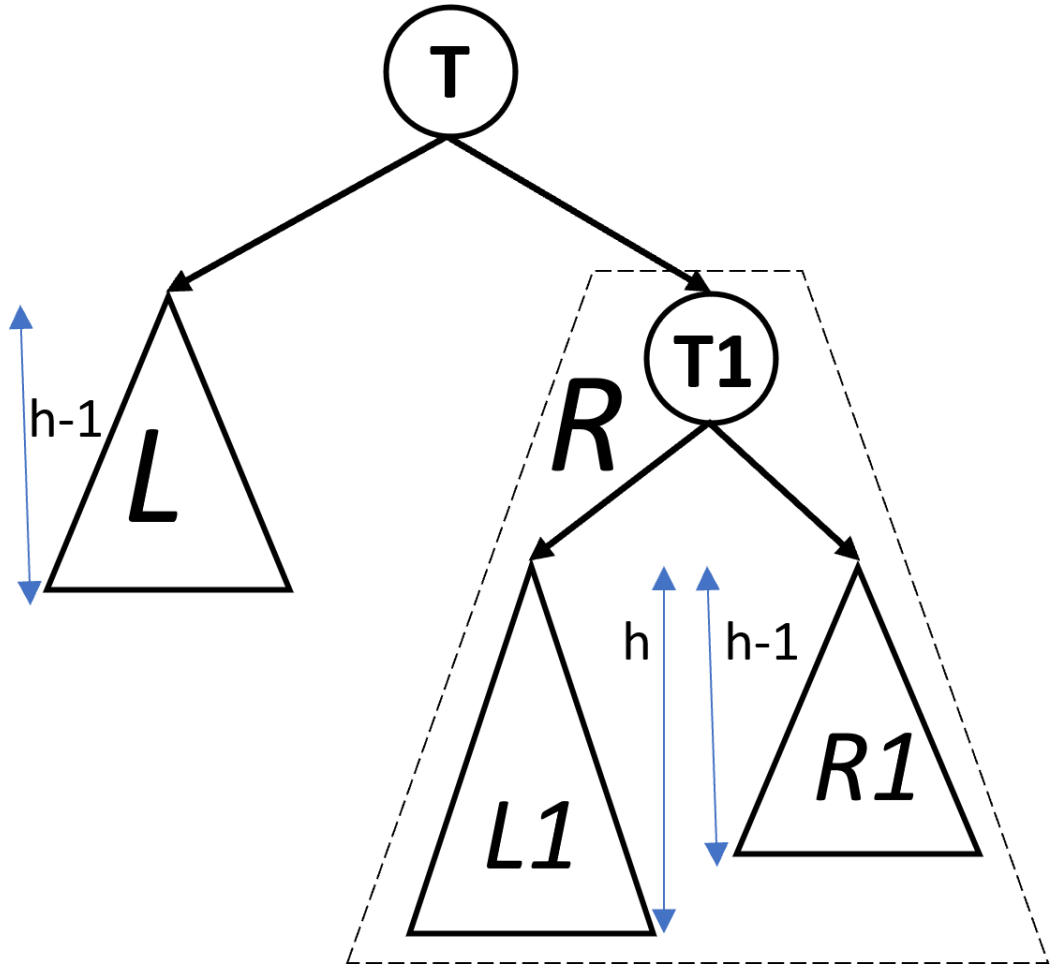
CÁCH XỬ LÝ:

- Ta thấy trường hợp này là đối xứng của trường hợp 1-2.
- Để cân bằng lại, ta làm các thao tác tương tự trường hợp 1-2 nhưng ngược hướng.
- Cụ thể là thực hiện phép quay đơn Right-Right.

TRƯỜNG HỢP 2 - CÂY LỆCH VỀ BÊN PHẢI

TRƯỜNG HỢP 2-3

- Là trường hợp cây mất cân bằng tại T và bị lệch về bên nhánh phải R.
- Nút T1 có chỉ số cân bằng là LH (tức là -1).



NHẬN XÉT:

- Nhánh R có chiều cao là h+1, nhánh L có chiều cao là h-1. Lệch nhau 2 đơn vị.
- Nhánh R cân bằng, có nút gốc là T1. Tại T1, nhánh trái cao hơn nhánh phải một đơn vị.

CÁCH XỬ LÝ:

- Trường hợp này là đối xứng của trường hợp 1-3.
- Do đó để cân bằng lại cây này, ta làm các thao tác tương tự trường hợp 1-3 nhưng ngược hướng.
- Cụ thể là thực hiện phép quay kép Right-Left.

CÂY NHỊ PHÂN TÌM KIẾM CÂN BẰNG

7.3 - HIỆN THỰC CÂY AVL

CÀI ĐẶT CÂY AVL

KHAI BÁO CẤU TRÚC MỘT NÚT CÓ CHỈ SỐ CÂN BẰNG

```
#define EH 0
#define RH 1
#define LH -1
```

```
typedef struct tnode
{    int key;
     char balFactor; //chỉ số cân bằng của nút
     struct tnode* pLeft;
     struct tnode* pRight;
}TNODE;
typedef TNODE* TREE;
```

CÀI ĐẶT CÂY AVL

Các tác vụ cần cài đặt:

- TÁC VỤ QUAY ĐƠN LEFT-LEFT
- TÁC VỤ QUAY ĐƠN RIGHT-RIGHT
- TÁC VỤ QUAY KÉP LEFT-RIGHT
- TÁC VỤ QUAY KÉP RIGHT- LEFT
- TÁC VỤ CÂN BẰNG KHI CÂY BỊ LỆCH TRÁI
- TÁC VỤ CÂN BẰNG KHI CÂY BỊ LỆCH PHẢI
- TÁC VỤ CHÈN NÚT VÀO CÂY AVL
- TÁC VỤ TÌM NÚT THẾ MẠNG
- TÁC VỤ XÓA MỘT NÚT

(xem code trong giáo trình)

CÂY NHỊ PHÂN TÌM KIẾM CÂN BẰNG

7.4 - BÀI TẬP CÂN BẰNG CÂY AVL

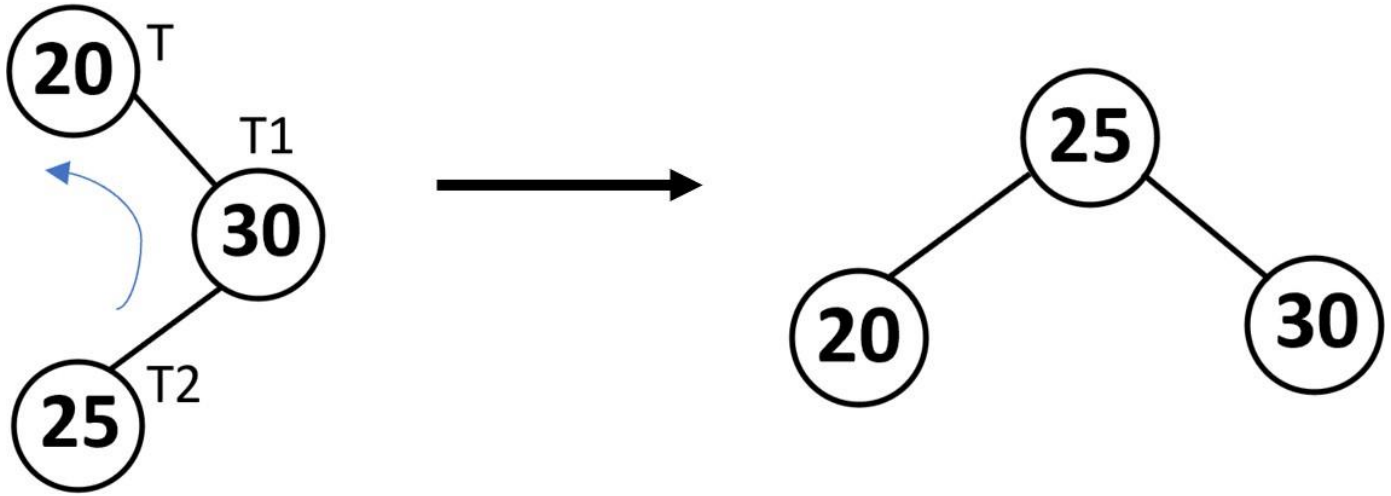
BÀI TẬP CÂN BẰNG CÂY AVL

- Hãy lần lượt sử dụng từng giá trị của dãy số sau đây để tạo cây AVL.

20 30 25 18 06 19 27 03 04

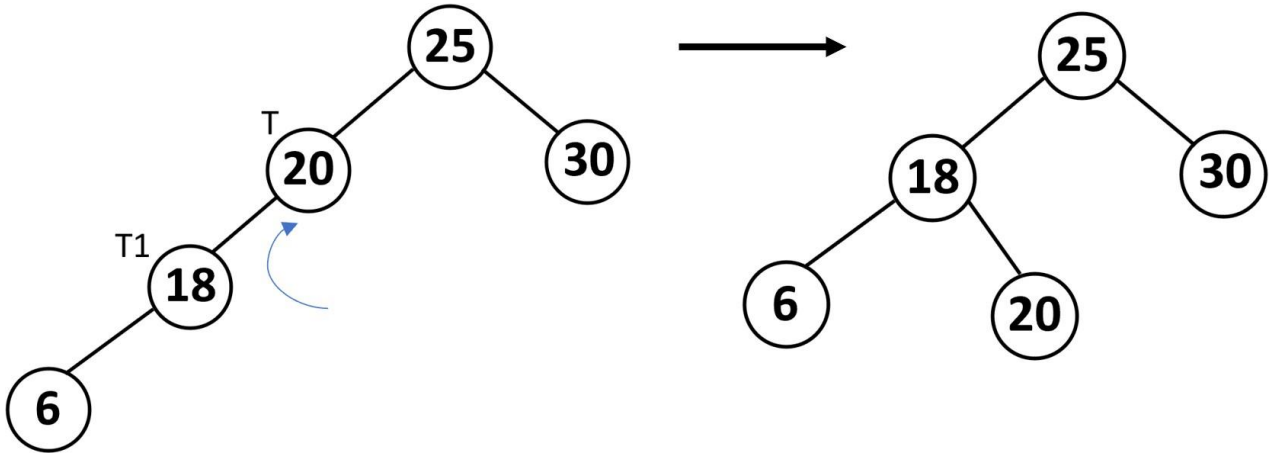
CÁC BƯỚC THỰC HIỆN NHƯ SAU:

- Thêm 20, rồi thêm 30.
- Thêm tiếp 25 thì cây mất cân bằng tại nút 20.
- Ta tiến hành quay cây.

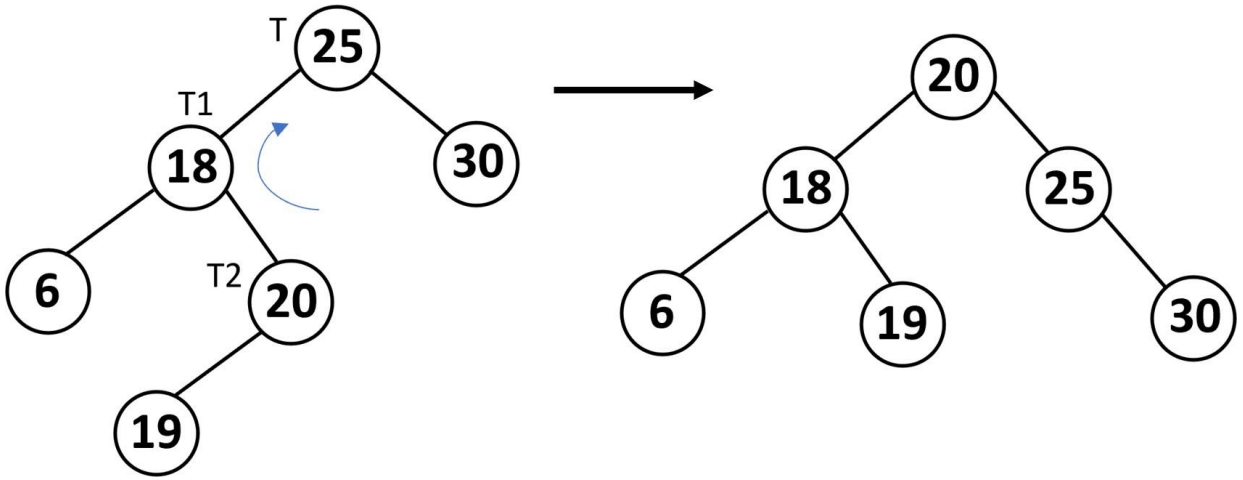


BÀI TẬP CÂN BẰNG CÂY AVL

- Thêm tiếp 18. Khi thêm 6 vào thì cây mất cân bằng tại nút 20. Ta tiến hành quay cây

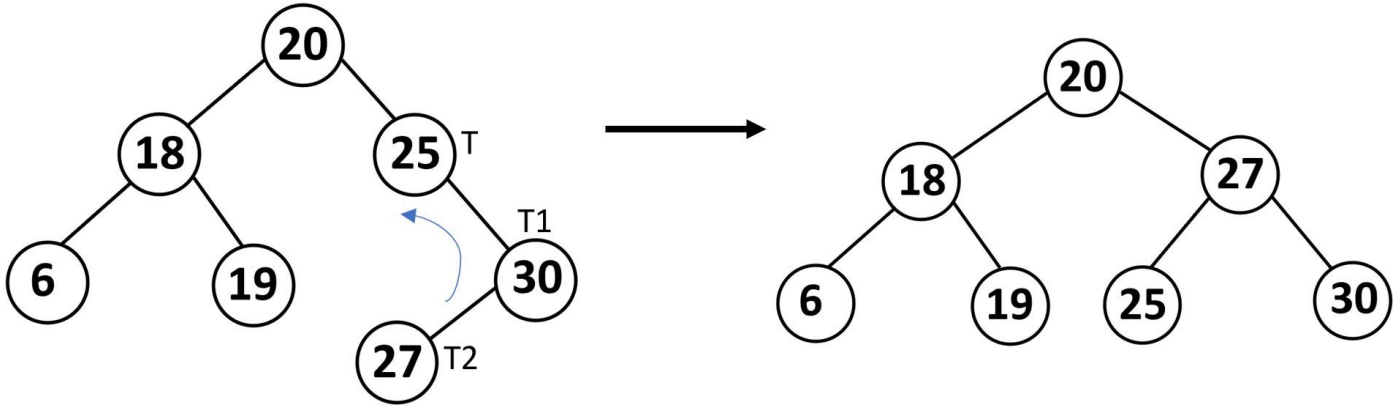


- Khi thêm 19 vào thì cây mất cân bằng tại nút 25. Ta tiến hành quay cây.

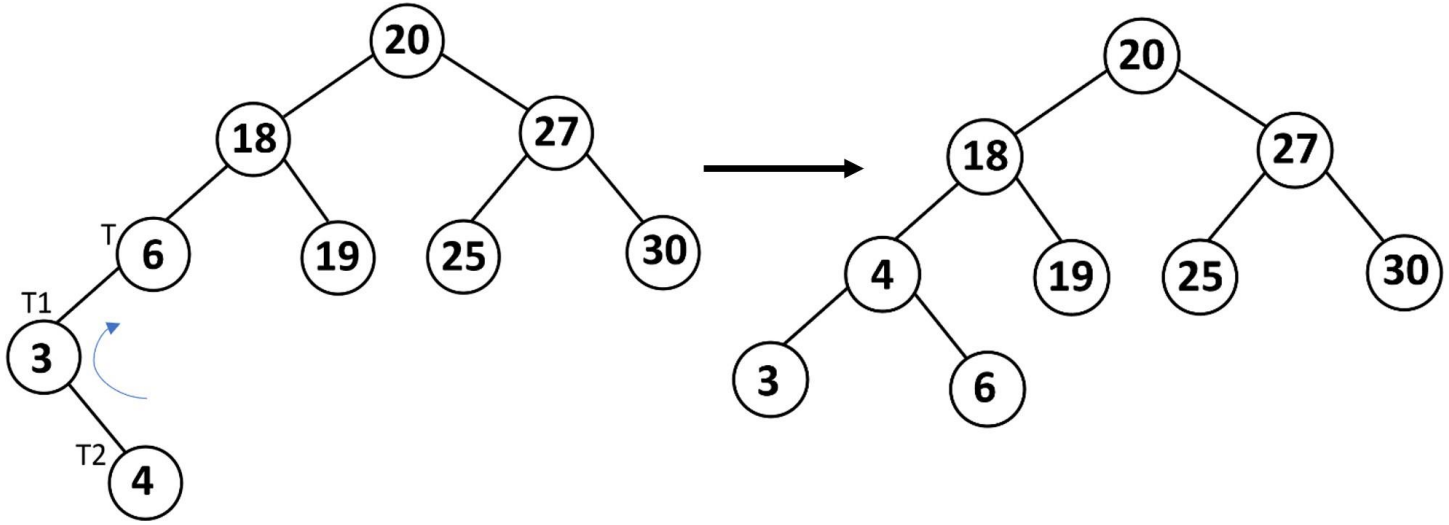


BÀI TẬP CÂN BẰNG CÂY AVL

- Khi thêm 27 vào thì cây mất cân bằng tại nút 25. Ta tiến hành quay cây.

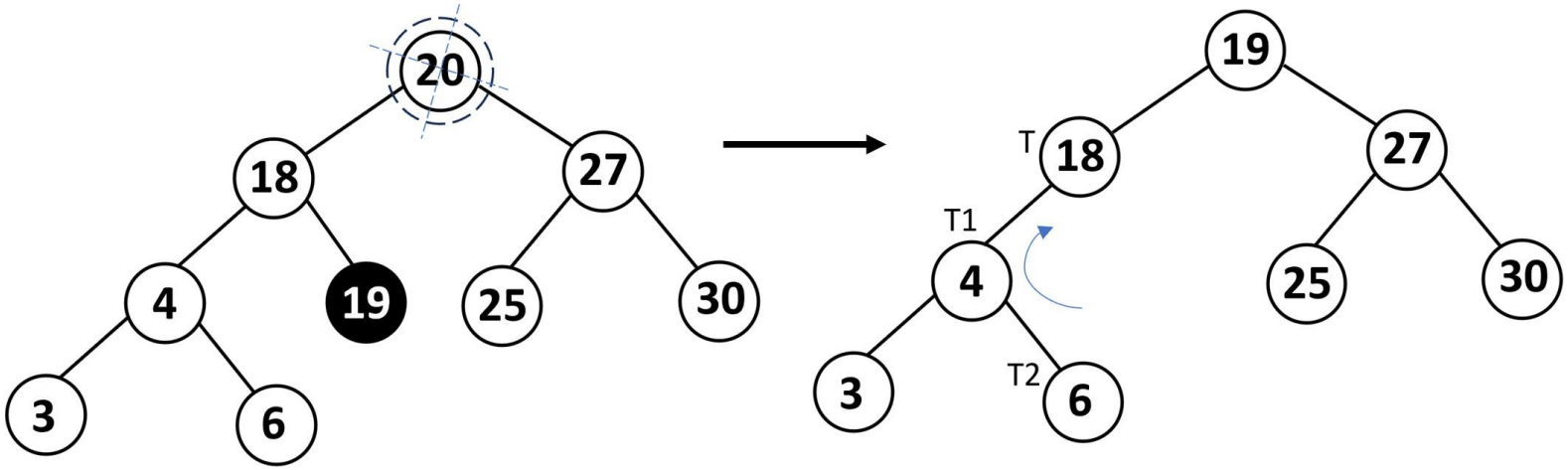


- Thêm tiếp 3 vào. Khi thêm 4 vào thì cây mất cân bằng tại nút 6. Ta tiến hành quay cây.

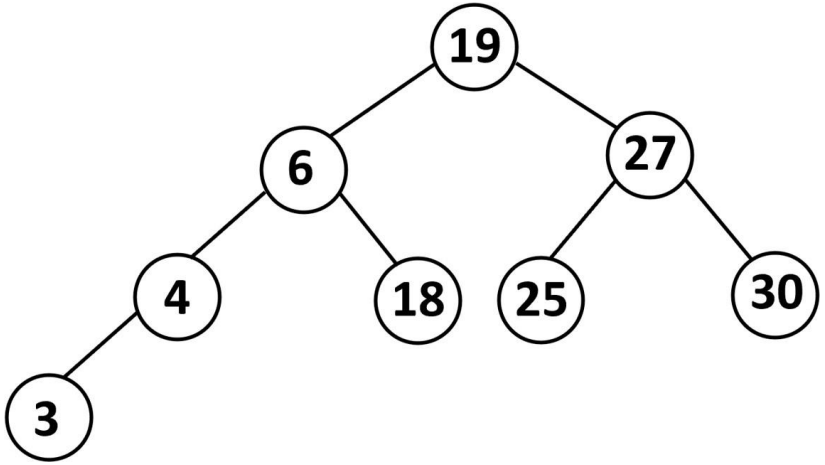


BÀI TẬP CÂN BẰNG CÂY AVL

- Bây giờ ta tiến hành xóa nút gốc 20, chọn nút 19 làm nút thế mạng



- Hậu quả là cây bị mất cân bằng tại nút 18. Ta tiến hành quay cây. Kết quả cuối cùng như sau.



BÀI TẬP

BÀI 1: Hãy cài đặt lại toàn bộ các tác vụ của cây AVL đã được hướng dẫn trong bài học. Viết hàm main gọi thực hiện một cách thích hợp các tác vụ.

BÀI 2: Hãy so sánh ưu điểm và nhược điểm giữa cây nhị phân tìm kiếm BST và cây nhị phân tìm kiếm cân bằng AVL.

BÀI 3: Hãy lần lượt sử dụng từng giá trị của dãy số sau đây để tạo cây AVL. Mỗi khi chèn một nút mới vào cây, nếu nhận thấy cây mất cân bằng thì phải tiến hành quay cây để khử sự mất cân bằng.

Dãy số: 15 80 08 06 05 07 02 82

Khi xây dựng cây xong thì tiến hành xóa nút 08. Nếu cây mất cân bằng thì tiến hành thực hiện các tác vụ quay cây thích hợp để cây cân bằng trở lại.

BÀI TẬP

BÀI 4: Hãy giải thích khái niệm cây AVL và nêu rõ các đặc điểm chính của nó. Tại sao cây AVL lại có hiệu suất tốt hơn so với cây nhị phân tìm kiếm (BST) thông thường trong việc thực hiện các thao tác tìm kiếm, chèn và xóa.

BÀI 5: Sử dụng cây AVL viết một chương trình như sau.

- Mô tả: Xây dựng một chương trình quản lý danh bạ điện thoại sử dụng cây AVL. Mỗi người trong danh bạ được lưu trữ dưới dạng một nút trong cây AVL, với số điện thoại là khóa (key) và tên người là giá trị tương ứng.

- Yêu cầu:
  - Cài đặt cây AVL để lưu trữ thông tin danh bạ.
  - Cung cấp các thao tác để thêm, xóa và tìm kiếm thông tin danh bạ dựa theo số điện thoại.
  - Viết hàm in danh sách các danh bạ theo thứ tự tăng dần của số điện thoại.

Q&A!